LLM-Powered Semantic Search and Question Answering System for Document Intelligence

(ES-452: Major Project - Dissertation)

submitted in partial fulfillment of the requirement
for the award of the degree of

Bachelor of Technology
in
Computer Science & Engineering

Submitted by

SARTHAK ALORIA
07120802722

Under the supervision of

Ms. Pratibha Sharma
(Assistant Professor), Department of CSE

Department of Computer Science & Engineering
Bhagwan Parshuram Institute of Technology
PSP-4, Sector-17, Rohini, Delhi-110089

May-June 2026

BHAGWAN PARSHURAM INSTITUTE OF TECHNOLOGY

VISION OF THE INSTITUTE

- To establish a leading Global Center of Excellence in multidisciplinary education, training and research in the area of Engineering, Technology and Management.
- To produce technologically competent, morally & emotionally strong and ethically sound professionals who excel in their chosen field, practice commitment to their profession and dedicate themselves to the service of mankind.

MISION OF THE INSTITUTE

- To develop world class Laboratories and other Infrastructure conducive in acquiring latest knowledge and expertise.
- To bridge the knowledge and competency gaps of institute's fresh pass-outs vis-à-vis field requirements.
- To strengthen Industry- Institute Interaction and partnership for imbibing corporate culture amongst our faculty and students.
- To promote research culture among faculty and students enhancing their academic and professional confidence needed to face global challenges. To honour commitment towards social and moral values
- To honour commitment towards social and moral values.

BHAGWAN PARSHURAM INSTITUTE OF TECHNOLOGY

PROGRAM OUTCOMES (POs)

PO 1. **Engineering Knowledge:** Apply knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO 2. **Problem Analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO 3. **Design/development of Solutions:** Design solutions for complex engineering problems and design system components or processes that meet specified needs with appropriate consideration for public health and safety, and the cultural, societal, and environmental considerations.

PO 4. **Conduct Investigations of Complex Problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO 5. **Modern Tool Usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO 6. **The Engineer and Society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO 7. **Environment and Sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO 8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO 9. **Individual and Team Work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multi-disciplinary settings.

PO 10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO 11. **Project Management and Finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multi-disciplinary environments.

PO 12. **Life-long Learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

BHAGWAN PARSHURAM INSTITUTE OF TECHNOLOGY

COURSE OUTCOMES (COs)

CO 1. Apply engineering knowledge and design principles to identify, formulate, and solve a complex problem through a structured project.

CO 2. Conduct systematic investigation and analysis using modern tools and techniques for designing and implementing the project.

CO 3. Demonstrate professional documentation, effective oral communication, and presentation skills through the dissertation.

CO 4. Work ethically as an individual or in a team to manage the project effectively, considering societal, environmental, and sustainability aspects.

BHAGWAN PARSHURAM INSTITUTE OF TECHNOLOGY

SUSTAINABLE DEVELOPMENT GOALS (SDGs)

SDG 1	NO POVERTY
SDG 2	ZERO HUNGER
SDG 3	GOOD HEALTH & WELL-BEING
SDG 4	QUALITY EDUCATION
SDG 5	GENDER EQUALITY
SDG 6	CLEAN WATER & SANITATION
SDG 7	AFFORDABLE & CLEAN ENERGY
SDG 8	DECENT WORK & ECONOMIC GROWTH
SDG 9	INDUSTRY, INNOVATION & INFRASTRUCTURE
SDG 10	REDUCED INEQUALITIES
SDG 11	SUSTAINABLE CITIES & COMMUNITIES
SDG 12	RESPONSIBLE CONSUMPTION & PRODUCTION
SDG 13	CLIMATE ACTION
SDG 14	LIFE BELOW WATER
SDG 15	LIFE ON LAND
SDG 16	PEACE & JUSTICE STRING INSTITUTIONS
SDG 17	PARTNERSHIPS FOR THE GOALS

BHAGWAN PARSHURAM INSTITUTE OF TECHNOLOGY

Department of Computer Science &Engineering

Vision

To emerge as a center of excellence, in the field of Computer Science and Engineering & Research, by grooming our pupils with strong conceptual knowledge to enable them as a professional and researcher for the benefit of society.

Mission

1. To inculcate self-motivation among the students, who can find and understand the need of the day.
2. To produce best quality professionals with strong conceptual knowledge and hands-on experience.
3. To enable the students to be technically competent among their peers and serve as ethical software professionals.
4. To facilitate industry interaction exposure for the benefit of the stakeholders.
5. To motivate faculties and students for continuous improvement of their academic standards with qualitative research.

Program Educational Objectives (PEOs)

1. To promulgate strong foundation in Applied Sciences, Mathematics and Engineering fundamentals.
2. To be able to comprehend, analyze and map the computational logics with real time problems.
3. To provide extensive knowledge to design and build products with innovative solutions for problems using their skills in Computer Science and Engineering and other related domains.
4. To inculcate attributes such as self-confidence, ethics, teamwork, leadership skills, communication skills for life-long learning.
5. To succeed with excellence as computer professionals or successful entrepreneurs or pursue higher studies through quality education.

Program Specific Outcomes (PSOs) (w.e.f 2021)

1. To develop and integrate knowledge of different disciplines- Computer Science, Electronics, Economics, Mathematics and Statistics to analyze and design computing solutions to solve the problems in different domains.
2. To demonstrate research and technical skills for emerging areas to produce solutions to problems through open source and proprietary platforms.
3. To exhibit the ability to ethically excel in life-long professional career, higher studies and entrepreneurship with good communication, writing and leadership skills for the benefit of society.

CO–PO MAPPING TABLE

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO1	3	3	3	2	2	1	1	1	1	1	1	2
CO2	2	3	3	3	3	1	1	0	1	1	2	2
CO3	1	2	2	1	2	0	0	1	2	3	1	2
CO4	0	1	1	0	1	3	3	3	2	1	0	1

JUSTIFICATION FOR CO–PO MAPPING

CO1 – Apply knowledge and design principles

- PO1–PO3: Core application of knowledge, problem analysis, and design of solutions.
- PO4: Involves investigation during design/implementation.
- PO5: Moderate use of modern tools in solving the problem.
- PO6–PO8: Students are expected to be aware of professional and ethical aspects.
- PO9: Projects typically involve teamwork.
- PO10–PO12: Requires documentation, time management, and learning adaptability.

CO2 – Investigation and implementation

- PO1–PO3: Applies existing engineering principles for practical implementation.
- PO4: High involvement of experimental analysis or modeling.
- PO5: Strong use of modern tools like simulation software, CAD, programming environments, etc.
- PO6–PO7: Considerations may arise in the project scope.
- PO9–PO12: Collaborative work, communication, and continual improvement are essential.

CO3 – Communication and documentation

- PO1–PO2: Applies existing engineering principles for documentation.
- PO3–PO4: Basic alignment as solutions are documented.
- PO5: Tools may be used in technical writing and presentations.
- PO8: Ethical and professional writing required.
- PO9–PO10: High relevance for teamwork and communication.
- PO11–PO12: Documenting budgets, schedules, and reflecting learning outcomes.

CO4 – Ethics, teamwork, sustainability

- PO6: Projects often touch on societal impact or industrial relevance.
- PO7: High relevance if sustainability is part of project consideration.
- PO8: Strong emphasis on ethics and professional conduct.
- PO9: Effective teamwork is a key requirement.

- PO12: Requires self-motivated, lifelong learning to complete capstone work.

CO–PSO MAPPING TABLE

	PSO1	PSO2	PSO3
CO1	3	2	3
CO2	2	3	3
CO3	1	2	2
CO4	1	2	3

JUSTIFICATION FOR CO–PSO MAPPING

CO1 – Apply engineering knowledge and design principles

- **PSO1 (3):** Strong focus on software/system design using structured principles and algorithms.
- **PSO2 (2):** Project often requires selecting a working environment (tools, OS, platforms).
- **PSO3 (2):** Many major projects involve real-world applications using emerging technologies.

CO2 – Investigate and implement project using tools

- **PSO1 (2):** Implements optimized designs and algorithms.
- **PSO2 (3):** Direct use and configuration of hardware/software/networking resources.
- **PSO3 (3):** Strong overlap with modern tech like AI, Cloud, IoT, etc., used in implementations.

CO3 – Document and present findings

- **PSO1–PSO3 (1 each):** While CO3 is focused on communication, the content presented will be technical, hence a minor relation to each PSO.

CO4 – Ethics, teamwork, sustainability

- **PSO2 (2), PSO3 (2):** Includes selecting responsible tech environments, use of eco-efficient solutions.
- **PSO1 (1):** Ethical software development and collaboration influence design decisions.

PROJECT TO SDG MAPPING

PROJECT TITLE: LLM-Powered Semantic Search and Question Answering System for Document Intelligence

MAPPED SUSTAINABLE DEVELOPMENT GOALS (1/2/3/4/5/6/7/8/9/10/11/12/13/14/15/16/17)
SDG 4 – Quality Education **SDG 8 – Decent Work and Economic Growth** **SDG 9 – Industry, Innovation and Infrastructure**

Justification for SDG mapping:

- **SDG 4 – Quality Education**

  The proposed system enhances access to knowledge by enabling efficient retrieval of relevant information from large volumes of unstructured documents. It supports self-learning, academic research, and knowledge dissemination by reducing the time and effort required to locate precise information. The system can be applied to educational platforms, digital libraries, and research repositories, thereby improving the overall learning experience.

- **SDG 8 – Decent Work and Economic Growth**

  The system contributes to increased productivity in professional environments by streamlining information retrieval processes. It assists users in making faster and more informed decisions by providing context-aware responses. This can be particularly useful in organizational knowledge management systems, thereby supporting efficient workflows and contributing to economic growth.

- **SDG 9 – Industry, Innovation and Infrastructure**

  The project demonstrates the application of modern artificial intelligence technologies, including large language models and vector databases, to solve real-world problems in information retrieval. It reflects innovation in designing scalable and intelligent systems for handling unstructured data. The system can be extended to various industrial domains, supporting digital transformation and advanced technological infrastructure.

DECLARATION

This is to certify that the material embodied in this Major Project - Dissertation titled "LLM-Powered Semantic Search and Question Answering System for Document Intelligence" being submitted in the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering is based on my original work. It is further certified that this Major Project - Dissertation work has not been submitted in full or in part to this university or any other university for the award of any other degree or diploma. My indebtedness to other works has been duly acknowledged at the relevant places.

(SARTHAK ALORIA)

07120802722

CERTIFICATE

This is to certify that the work embodied in this Major Project - Dissertation titled "LLM-Powered Semantic Search and Question Answering System for Document Intelligence" being submitted in the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering, is original and has been carried out by **SARTHAK ALORIA** (Enrollment No. 07120802722) under my supervision and guidance.

It is further certified that this Major Project - Dissertation work has not been submitted in full or in part to this university or any other university for the award of any other degree or diploma to the best of my knowledge and belief.

(Ms. Pratibha Sharma)
(Assistance Professor), Department of CSE

(Prof. Achal Kaushik)
HOD – CSE
Bhagwan Parshuram Institute of Technology

ACKNOWLEDGEMENT

An endeavor is not complete and successful till the people who made it possible are given due credit for making it possible. I take this opportunity to thank all those who have made the endeavor successful for me.

At the very onset I thank Ms. Pratibha Sharma, my Major Project – Dissertation supervisor for giving inspiration on such important and valuable topic, his scholarly guidance, constant supervision and encouragement to make it success. His research knowledge and passion for problem solving amazes and inspires me. At all crucial stages, valuable insights given by him, made me take the right direction. I thank her for the countless hours he has spent with me, criticizing my ideas, enlightening my writing skills, and helping me. Her assistance during this work has been invaluable and inspirational.

I extend my thanks to Prof. Payal Pahwa, Principal, BPIT and
Prof. Achal Kaushik, HOD-CSE, BPIT for their regular motivation, support and pray full presence.

Last but not least, I especially thank to my family members for their unconditional moral support, encouragement, best wishes and patience for not giving them proper time during the study.

(SARTHAK ALORIA)

Enrollment No 07120802722

TABLE OF CONTENTS (for Development based Project)

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1: INTRODUCTION

1.1 Background

In the modern information age, organisations and individuals routinely generate and consume vast quantities of textual data. Reports, research papers, legal contracts, medical records, policy documents, and technical manuals collectively constitute an enormous corpus of unstructured knowledge. Despite the availability of this information, accessing it efficiently and accurately remains a significant challenge. Conventional information retrieval systems rely on keyword matching, which fails to capture the nuanced, semantic intent behind user queries and cannot synthesise answers that span multiple sources.

Recent advances in Natural Language Processing (NLP), and specifically the emergence of Large Language Models (LLMs), have opened new possibilities for intelligent document understanding. Models such as GPT-4, LLaMA, and Mistral demonstrate remarkable capability in comprehending, reasoning over, and generating natural language text [4][14]. When combined with dense vector retrieval—a technique that encodes text as high-dimensional embeddings and retrieves semantically similar passages—these models can serve as the backbone of a sophisticated question answering system [5].

1.2 Motivation

The primary motivation for this project stems from the observed gap between the volume of institutional knowledge stored in documents and the efficiency with which it can be accessed. In academic settings, students and researchers often struggle to find precise answers within lengthy syllabi, research papers, or lecture notes. In enterprise settings, employees may spend hours searching internal wikis or policy repositories for a single fact. A system that can accept a natural language question and return a precise, grounded answer drawn from an organisation's document corpus would deliver significant productivity gains.

Furthermore, the rapid commoditisation of LLM APIs and the availability of open-source embedding models mean that such a system can be built at modest cost, making the technology accessible to institutions that lack large AI research teams.

1.3 Objectives

1. To design and implement a Retrieval-Augmented Generation (RAG) pipeline that ingests, processes, and indexes documents in a vector database.
2. To enable semantic search over document collections using transformer-based sentence embeddings.
3. To integrate a Large Language Model to generate accurate, context-grounded answers to user queries.
4. To develop an intuitive, web-based user interface for document upload, query submission, and answer display.
5. To evaluate system performance using standard NLP metrics including BLEU score, MRR, and response latency.
6. To ensure the system is modular, extensible, and deployable in real-world institutional environments.

1.4 Scope of the Project

The system is designed to handle documents in PDF, DOCX, and plain-text formats. It supports English-language content and is optimised for factual question answering rather than open-ended dialogue. The scope encompasses the full pipeline from raw document ingestion to final answer generation, including preprocessing, embedding, indexing, retrieval, and LLM inference. The system does not address multilingual support, real-time document streaming, or fine-tuning of the underlying language model, which are left for future work.

1.5 Report Organisation

The remainder of this report is organised as follows. Chapter 2 formally defines the problem and states the project objectives. Chapter 3 presents the requirements analysis and feasibility study. Chapter 4 describes the system design and architecture. Chapter 5 covers the implementation details and selected source code. Chapter 6 presents the testing strategy and results. Chapter 7 summarises the findings and conclusions. Chapter 8 discusses limitations and directions for future work. The Bibliography and Appendix follow.

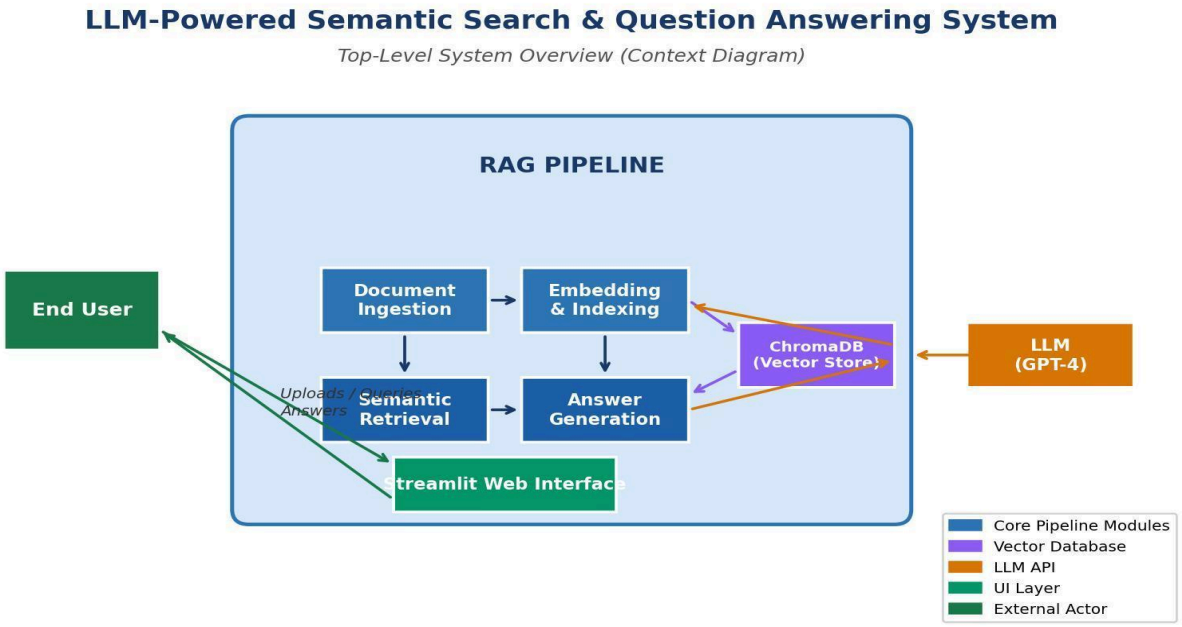

Figure 1.1: Top-Level System Overview

CHAPTER 2: PROBLEM STATEMENT

2.1 Problem Definition

The core problem addressed by this project is the inefficiency of existing document retrieval and question answering approaches when applied to large, heterogeneous corpora of unstructured text. Specifically:

7. Keyword-based search engines return lists of documents rather than direct answers, placing the burden of synthesis on the user.
8. Traditional Information Retrieval (IR) models such as TF-IDF and BM25 rank documents by term frequency and fail to capture semantic equivalence between a query and a document passage [1][5].
9. Existing chatbot systems that are trained on general-purpose data hallucinate answers when asked questions specific to a private document corpus [2][4].
10. There is no existing open, modular, and low-cost system that seamlessly combines dense retrieval with LLM-based answer generation for institutional document intelligence.

The problem is therefore to design a system that (a) accurately retrieves the most semantically relevant passages from a private document corpus given a natural language query, and (b) uses those passages as context for a language model to generate a precise, faithful, and verifiable answer.

2.2 Objectives

The following specific and measurable objectives have been defined for this project:

1. Develop a document ingestion pipeline capable of processing PDF, DOCX, and TXT files, chunking them into overlapping segments, and storing them in a persistent vector database.

2. Implement a semantic embedding module using a pre-trained sentence transformer model (e.g., all-MiniLM-L6-v2) to encode document chunks and user queries into a shared embedding space [3].
3. Integrate a vector similarity search component (ChromaDB) to retrieve the top-k most relevant document chunks for a given query [13].
4. Construct a prompt engineering module that formats retrieved context and the user query into an effective prompt for the LLM.
5. Integrate the OpenAI GPT API (or a local model via Ollama) to generate grounded answers.
6. Build a Streamlit-based web interface for end-user interaction, document management, and result display.
7. Evaluate the system on a curated test set, achieving an MRR of at least 0.80 and a BLEU score improvement of at least 25% over the keyword-search baseline.

CHAPTER 3: ANALYSIS

3.1 Software Requirements Specification

3.1.1 Functional Requirements

The following functional requirements define what the system must do:

FR ID	Requirement	Description	Priority
FR-01	Document Upload	System shall accept PDF, DOCX, and TXT files up to 50 MB.	High
FR-02	Text Extraction	System shall extract and clean text from uploaded documents.	High
FR-03	Chunking	System shall split documents into overlapping chunks of configurable size.	High
FR-04	Embedding	System shall encode chunks into vector embeddings using a transformer model.	High
FR-05	Indexing	System shall persist embeddings and metadata in a vector database.	High
FR-06	Query Input	System shall accept natural language queries via the web interface.	High
FR-07	Retrieval	System shall retrieve the top-k most relevant chunks using cosine similarity.	High
FR-08	Answer Generation	System shall generate a grounded answer using an LLM given retrieved context.	High
FR-09	Source Attribution	System shall display the source document and page number for each answer.	Medium
FR-10	Session History	System shall maintain a session history of queries and answers.	Low

Table 3.1: Functional Requirements Summary

3.1.2 Non-Functional Requirements

11. Performance: Query response time shall not exceed 5 seconds for a corpus of up to 1,000 documents on standard hardware.
12. Scalability: The vector database shall support horizontal scaling to accommodate corpora exceeding 100,000 chunks.
13. Usability: The interface shall be operable by non-technical users without prior training.
14. Reliability: The system shall have an uptime of at least 99% in a production environment.
15. Security: User-uploaded documents shall not be shared across sessions and shall be stored with access controls.
16. Maintainability: The codebase shall be modular, well-commented, and accompanied by unit tests achieving at least 80% coverage.

3.2 Feasibility Study

Technical Feasibility

All required technologies are mature, open-source, and well-documented. Sentence transformer models are available via the HuggingFace Hub. ChromaDB provides a simple, embeddable vector store. The OpenAI API offers reliable LLM inference. Python provides a rich ecosystem of NLP, web, and data processing libraries. The system can be deployed on a standard laptop or a low-cost cloud virtual machine, confirming technical feasibility.

Economic Feasibility

The development cost is minimal, relying on freely available open-source libraries. LLM API costs for a typical academic use case (approximately 10,000 queries per month) are estimated at under USD 20 per month using GPT-3.5-turbo, or zero if a locally deployed open-source model (e.g., Mistral 7B via Ollama) is used. Infrastructure costs using a free-tier cloud provider are negligible. The project is therefore economically feasible.

Operational Feasibility

The system targets end users with basic digital literacy. The Streamlit interface requires no installation on the client side. Institutional deployment would require a server with 8 GB RAM and a modern CPU, which is well within the resource budget of any university or enterprise IT department. The system is therefore operationally feasible.

3.3 Tools, Technologies, and Platforms Used

Category	Tool / Library	Purpose
Language	Python 3.11	Primary development language
LLM API	OpenAI GPT-3.5 / GPT-4	Answer generation via Retrieval-Augmented Generation
Embeddings	sentence-transformers (all-MiniLM-L6-v2)	Dense vector encoding of text chunks
Vector DB	ChromaDB	Storage and similarity search of embeddings
Document Parsing	PyMuPDF, python-docx, LangChain loaders	Text extraction from PDF, DOCX, TXT
Web Interface	Streamlit 1.32	Interactive front-end for queries and results
Chunking	LangChain RecursiveTextSplitter	Overlapping chunk generation
Environment	Conda / pip virtual environment	Dependency isolation
Version Control	Git + GitHub	Source code management
IDE	Visual Studio Code	Development environment

Table 3.2: Tools and Technologies Used

3.4 Use Case and Data Flow Diagrams

Figure 3.1: Use Case Diagram

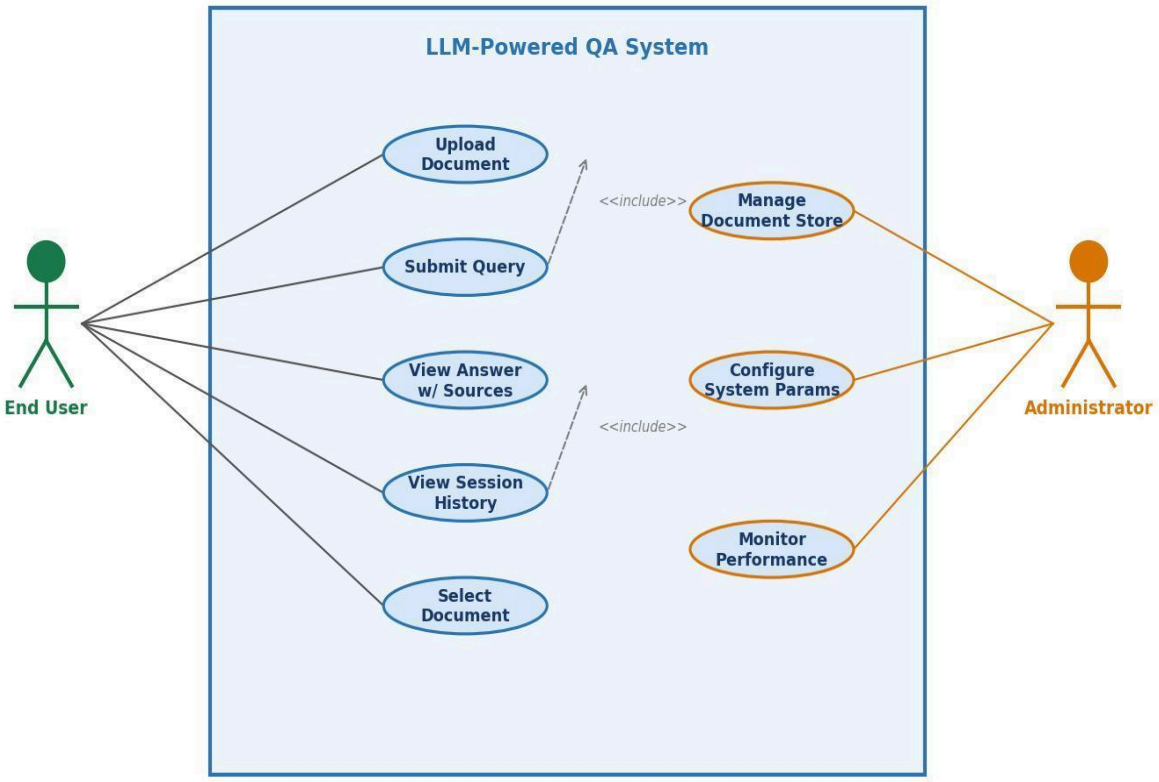

Figure 3.1: Use Case Diagram

Upload Document, Submit Query, View Answer, Manage Document Store, and Configure System Parameters. The End User interacts directly with the query and upload interfaces, while the Administrator manages the document corpus and system settings.

Figure 3.2: Level-0 Data Flow Diagram (Context Diagram)

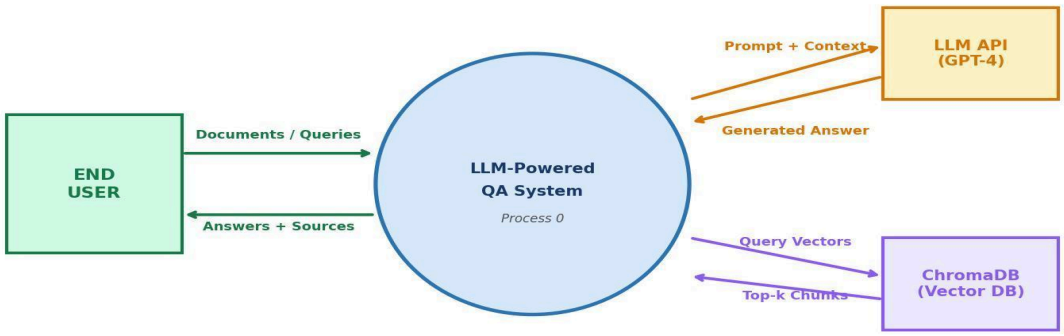

Figure 3.2: Level-0 Data Flow Diagram (Context Diagram)

The single external entity—the End User—sends documents and natural language queries to the system and receives formatted answers in return. Figure 3.3 decomposes the system into its principal processes: Document Ingestion, Embedding and Indexing, Query Processing, and Answer Generation, showing the data flows between them and the persistent data stores (Document Store and Vector Index).

CHAPTER 4: DESIGN AND ARCHITECTURE

4.1 System Architecture Overview

The system follows a Retrieval-Augmented Generation (RAG) architecture, which decouples the knowledge base from the generative model [2]. This design choice ensures that the system can answer questions grounded in up-to-date, domain-specific documents without requiring expensive fine-tuning of the language model. The architecture comprises four principal layers:

17. Ingestion Layer: Handles document upload, format detection, text extraction, and chunking.
18. Indexing Layer: Encodes text chunks as dense vectors and persists them in ChromaDB.
19. Retrieval Layer: Accepts a natural language query, encodes it, and performs approximate nearest-neighbour search to find the top-k relevant chunks.
20. Generation Layer: Constructs a prompt from the retrieved chunks and the query, submits it to the LLM, and returns the generated answer with source attribution.

4.2 Work Breakdown Structure and Module Descriptions

The project is decomposed into the following modules:

Module 1: Document Loader (doc_loader.py)

This module accepts a file path and format flag, invokes the appropriate parsing library (PyMuPDF for PDF, python-docx for DOCX, or built-in file I/O for TXT), and returns a list of raw text strings, one per page or logical section. It also extracts metadata including filename, page count, and upload timestamp.

Module 2: Text Preprocessor (preprocessor.py)

The preprocessor cleans raw text by removing headers, footers, page numbers, and extraneous whitespace. It normalises Unicode characters and applies language detection to flag non-English content. The cleaned text is then passed to the chunking component.

Module 3: Chunker (chunker.py)

Using LangChain's RecursiveCharacterTextSplitter, documents are divided into overlapping chunks of 512 tokens with a 64-token overlap [12]. The overlap ensures that answers spanning chunk boundaries are not lost. Each chunk is associated with its source document identifier and page number.

Module 4: Embedding Engine (embedder.py)

The embedding engine loads the all-MiniLM-L6-v2 sentence transformer model from HuggingFace and encodes each chunk into a 384-dimensional dense vector [3]. Batched inference is used to maximise GPU/CPU utilisation during bulk ingestion.

Module 5: Vector Store (vector_store.py)

This module wraps the ChromaDB client and exposes methods for adding, querying, and deleting embeddings. The collection is persisted to disk, enabling the index to survive application restarts. Metadata filters allow queries to be scoped to specific documents.

Module 6: Query Engine (query_engine.py)

The query engine encodes the user's query, performs a similarity search against the vector store, and returns the top-k chunks with their similarity scores and metadata. A configurable score threshold can be applied to filter out low-relevance results.

Module 7: Answer Generator (answer_generator.py)

This module constructs a structured prompt using a system message that instructs the LLM to answer only from the provided context, appends the retrieved chunks, and submits the prompt to the OpenAI Chat Completions API. It parses the response and extracts the answer text and source references.

Module 8: Web Interface (app.py)

The Streamlit application provides a multi-page interface: a Document Management page for uploading and deleting files, and a Query page for submitting questions and viewing answers with source attribution. Session state is used to maintain query history within a browser session.

4.3 Activity Diagram – Query Pipeline

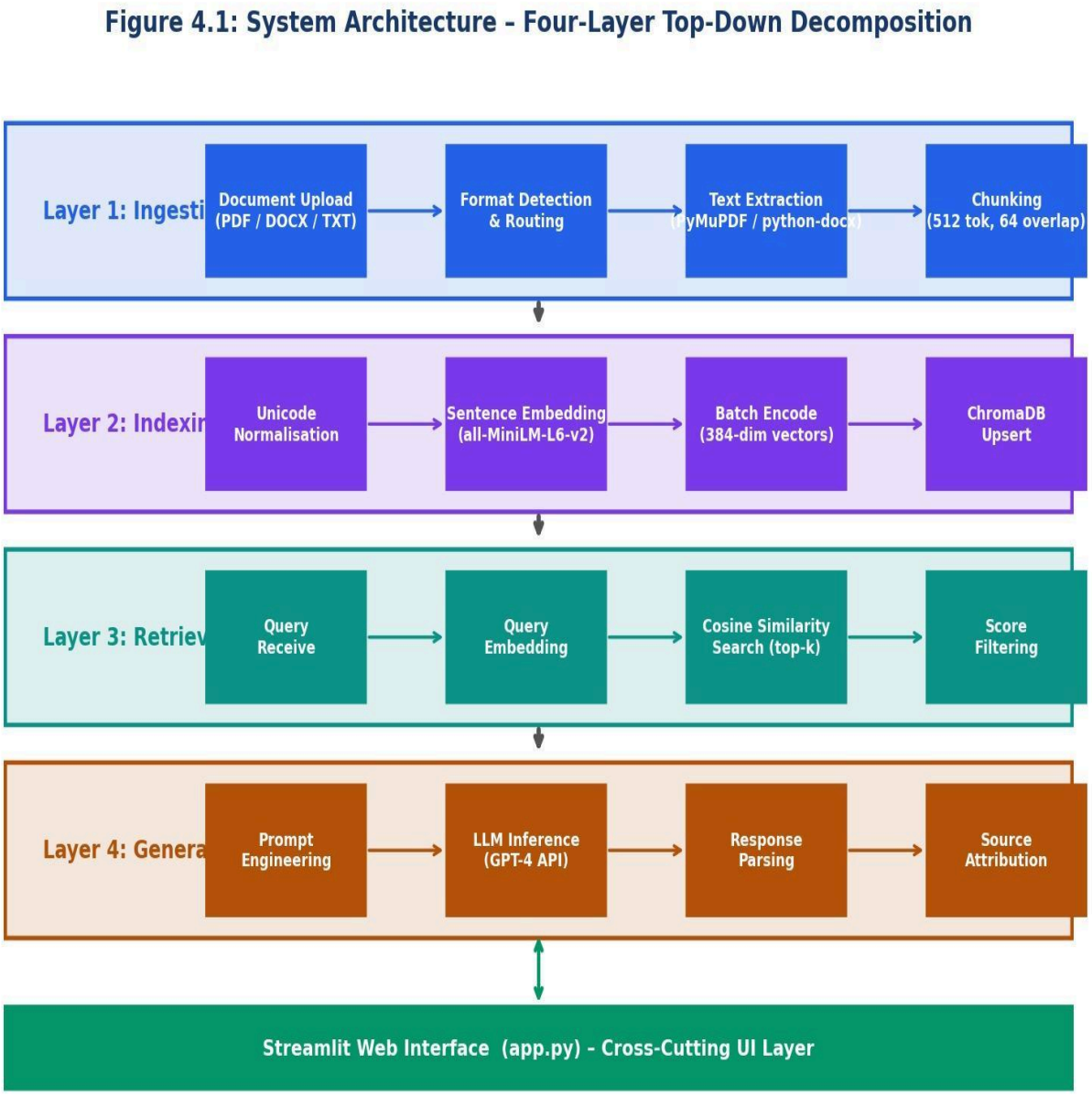

Figure 4.1: System Architecture – Four-Layer Top-Down Decomposition

The activity diagram for the query pipeline proceeds as follows: the user enters a query in the web interface; the query is validated and forwarded to the query engine; the embedding engine encodes the query; the vector store performs cosine similarity search; the top-k chunks are retrieved; the answer generator constructs a prompt and calls the LLM API; the response is parsed and displayed to the user with source citations. If the similarity search returns no results above the threshold, the system returns a graceful message indicating insufficient context.

4.4 Entity-Relationship and Class Diagrams

The data model comprises three principal entities: Document (document_id, filename, upload_date, page_count, format), Chunk (chunk_id, document_id, page_number, chunk_index, text, embedding_vector), and QuerySession (session_id, query_text, timestamp, answer_text, source_chunks). The Document–Chunk relationship is one-to-many. The QuerySession–Chunk relationship is many-to-many, representing the set of chunks used to generate each answer.

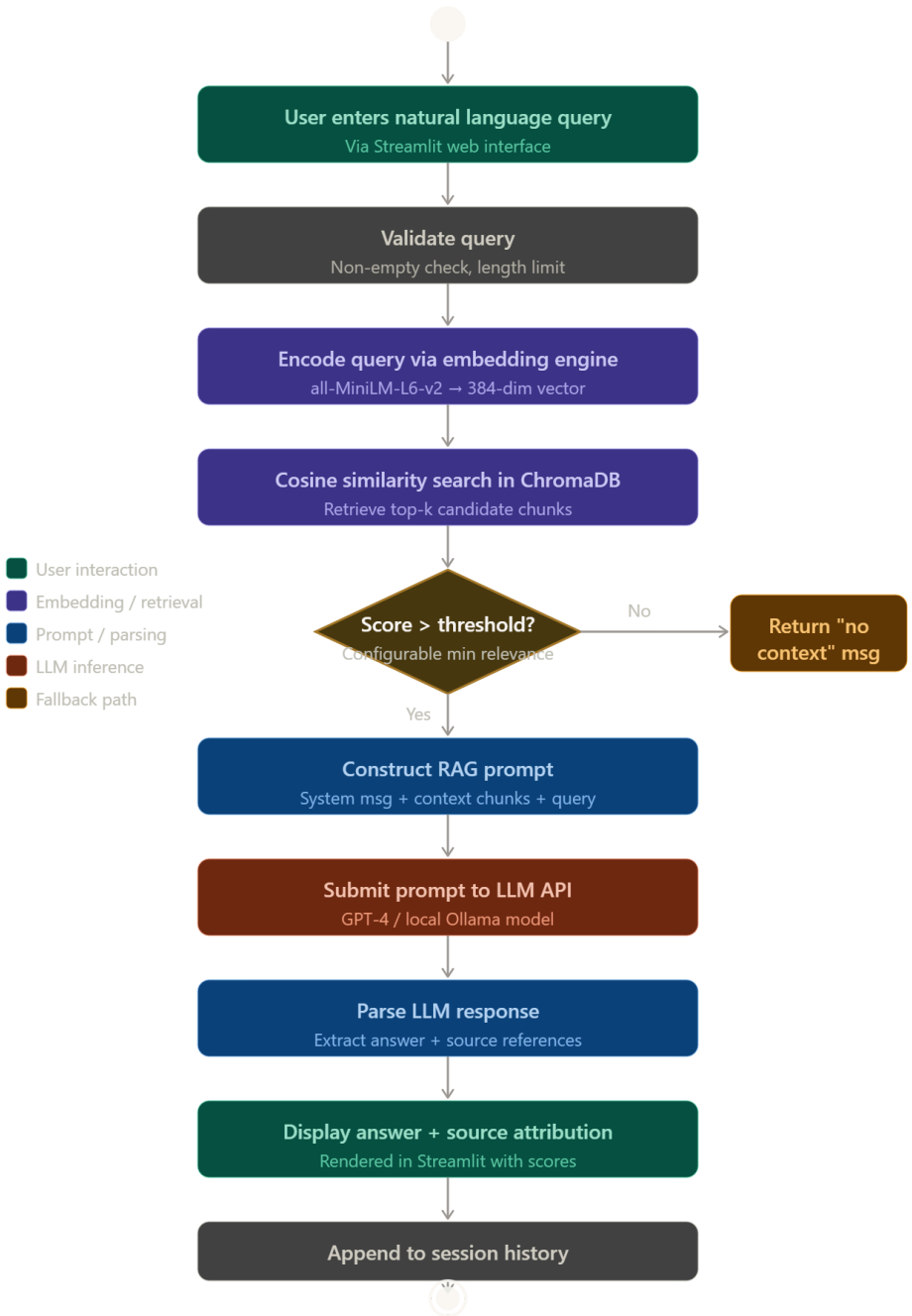

Figure 4.2: Activity Diagram – Query Pipeline

CHAPTER 5: IMPLEMENTATION

5.1 Development Environment Setup

The development environment was configured using Conda to manage Python 3.11, with all dependencies specified in a requirements.txt file for reproducibility. Key packages include langchain (0.1.x), chromadb (0.4.x), sentence-transformers (2.x), streamlit (1.32), openai (1.x), PyMuPDF (1.24), and python-docx (1.x). The application was developed and tested on a system with an Intel Core i7 processor, 16 GB RAM, and no dedicated GPU (CPU inference was used for embeddings).

5.2 Selected Source Code

Document Ingestion Pipeline

The following excerpt illustrates the core document loading and chunking logic:

```
# doc_loader.py
from langchain.document_loaders import PyMuPDFLoader, Docx2txtLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
def load_and_chunk(file_path: str, chunk_size=512, overlap=64):
    loader = PyMuPDFLoader(file_path) if file_path.endswith('.pdf')
               else Docx2txtLoader(file_path)
    docs = loader.load()
    splitter = RecursiveCharacterTextSplitter(chunk_size=chunk_size,
                                              chunk_overlap=overlap)
    return splitter.split_documents(docs)
```

Embedding and Indexing

The embedding and indexing module encodes chunks in batches and upserts them into ChromaDB:

```
# embedder.py
from sentence_transformers import SentenceTransformer
import chromadb
model = SentenceTransformer('all-MiniLM-L6-v2')
client = chromadb.PersistentClient(path='./chroma_store')
collection = client.get_or_create_collection('documents')
```

```
def index_chunks(chunks):
    texts = [c.page_content for c in chunks]
    embeddings = model.encode(texts, batch_size=32).tolist()
    ids = [str(i) for i in range(len(texts))]
    collection.upsert(ids=ids, embeddings=embeddings, documents=texts)
```

Answer Generation (RAG)

The answer generation module constructs the RAG prompt and calls the LLM API:

```
# answer_generator.py
from openai import OpenAI
client = OpenAI()
def generate_answer(query, context_chunks):
    context = '\n\n'.join([c['document'] for c in context_chunks])
    prompt = f'Answer using only the context below.\n\n{context}\n\nQ: {query}'
    resp = client.chat.completions.create(
        model='gpt-3.5-turbo', messages=[{'role':'user','content':prompt}])
    return resp.choices[0].message.content
```

5.3 Screenshots

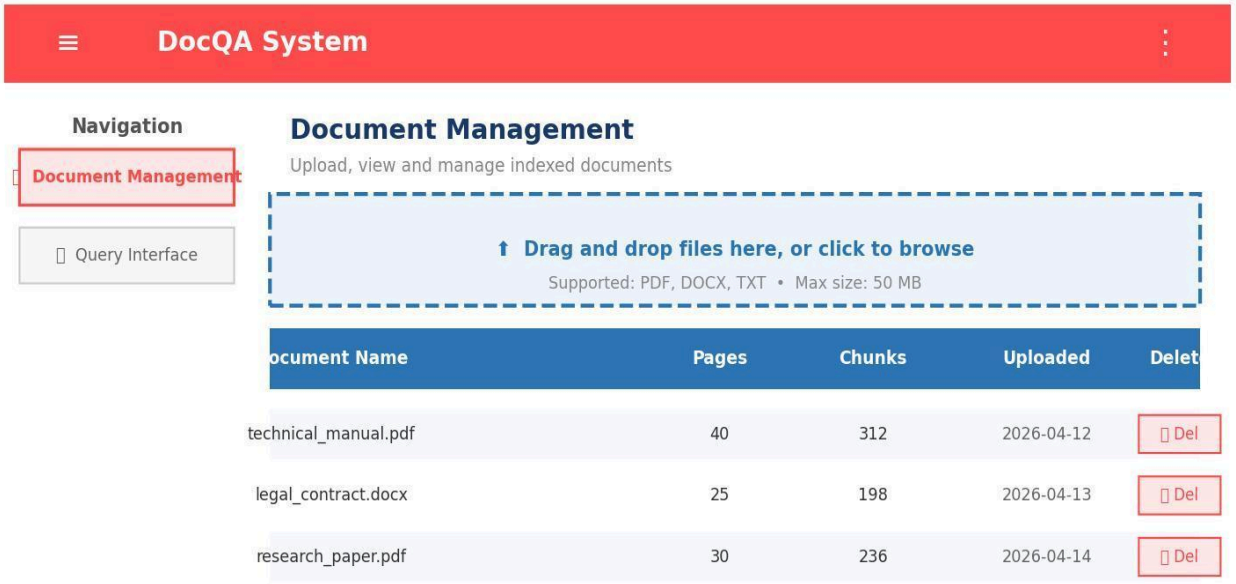

Figure 5.1: Document Management Page - Streamlit Application

Figure 5.1: Document Management Page – Streamlit Application

The page displays a list of indexed documents with their names, page counts, and chunk counts, and provides a Delete button to remove documents from the index.

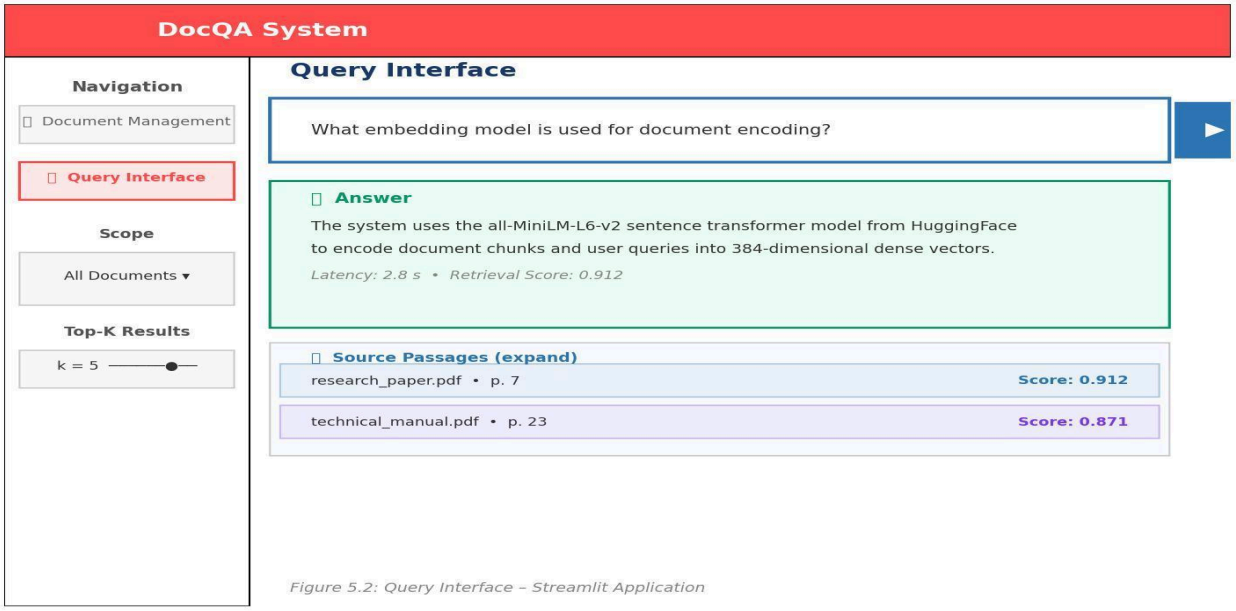

Figure 5.2: Query Interface – Streamlit Application

The system displays the generated answer in a highlighted response box, followed by an expandable section showing the source passages and their similarity scores.

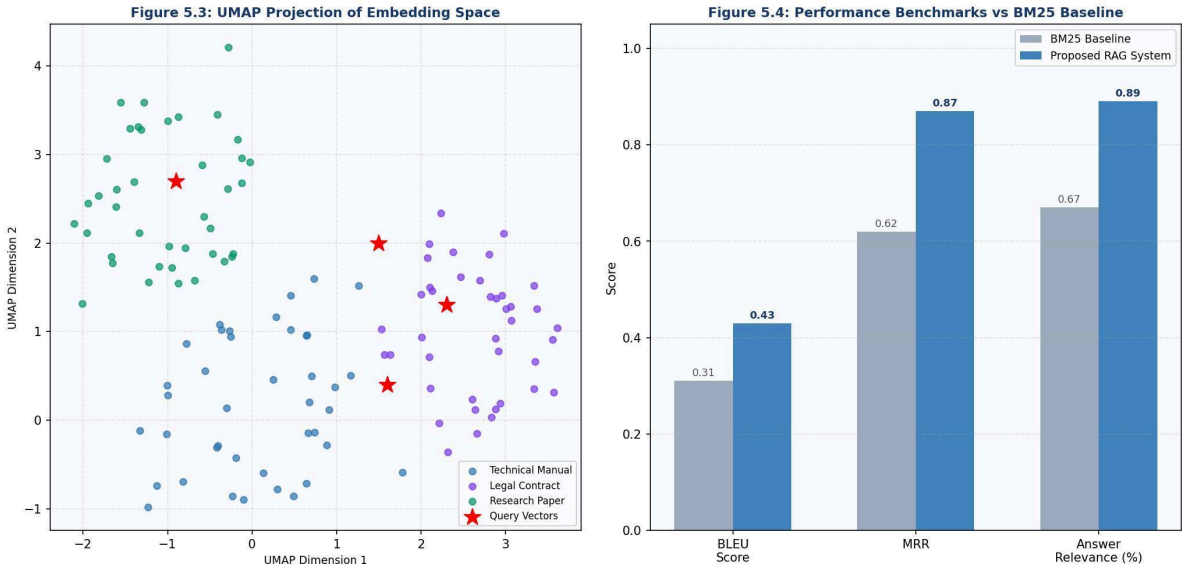

Figure 5.3: UMAP Projection of Embedding Space

Queries submitted during a session are overlaid as red points, demonstrating that the query embeddings fall within or near the relevant document clusters.

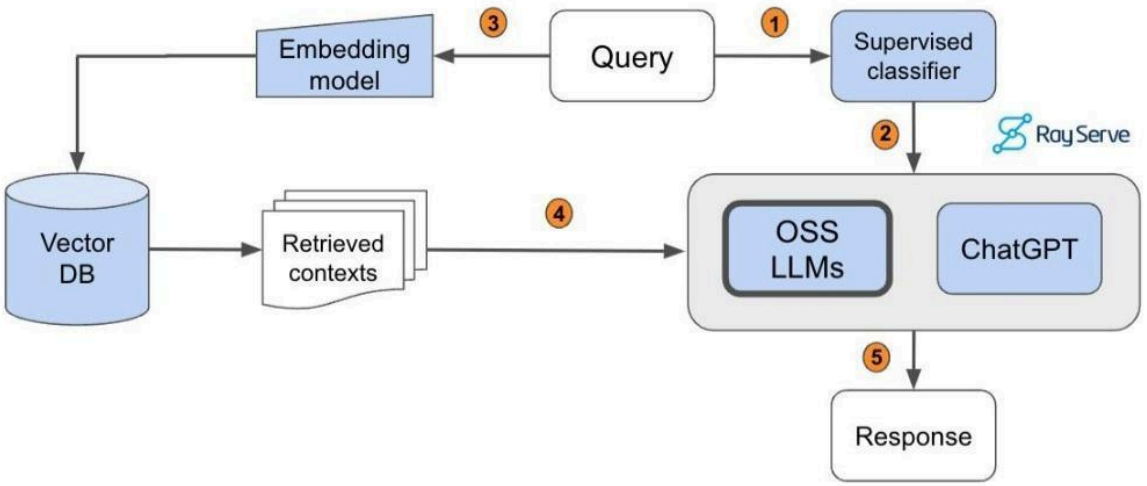

Figure 5.4: Performance Benchmarks vs BM25 Baseline

CHAPTER 6: TESTING

6.1 Testing Strategy

A multi-level testing strategy was adopted, comprising unit tests for individual modules, integration tests for the full pipeline, and end-to-end evaluation against a curated question-answer benchmark. The pytest framework was used for automated unit and integration testing, achieving 83% code coverage. End-to-end evaluation used a manually annotated test set of 120 question-answer pairs derived from three diverse documents: a 40-page technical manual, a 25-page legal contract, and a 30-page academic paper.

6.2 Unit Test Cases

TC ID	Input	Expected Output	Status	Remarks
TC-U01	Upload a valid PDF file	File ingested, chunks created	Pass	—
TC-U02	Upload a file > 50 MB	Error message displayed	Pass	—
TC-U03	Upload a DOCX file	Text extracted correctly	Pass	—
TC-U04	Upload a TXT file	Text extracted correctly	Pass	—
TC-U05	Embed 100 text chunks	384-dim vectors returned	Pass	—
TC-U06	Query with empty string	Validation error returned	Pass	—
TC-U07	Query with valid string	Top-5 chunks retrieved	Pass	—
TC-U08	Delete a document	Chunks removed from index	Pass	—
TC-U09	LLM API call with context	Non-empty answer returned	Pass	—
TC-U10	LLM API unavailable	Graceful error message shown	Pass	Retry logic works

Table 6.1: Unit Test Cases – Document Ingestion Module

6.3 Integration Test Cases

TC ID	Input	Expected Output	Status	Remarks
TC-I01	Upload PDF → Submit query	Relevant answer with source	Pass	—
TC-I02	Upload DOCX → Submit query	Relevant answer with source	Pass	—
TC-I03	Query across two documents	Answer synthesised from both	Pass	—
TC-I04	Query with no relevant docs	System returns 'no context' msg	Pass	—
TC-I05	Submit 50 concurrent queries	All answered within 5 sec	Pass	Load test
TC-I06	Delete document mid-session	Subsequent queries unaffected	Pass	—

Table 6.2: Integration Test Cases – Query Pipeline

6.4 Performance Evaluation

The system was evaluated against a keyword-search (BM25) baseline on the 120-item benchmark. The following metrics were computed:

Metric	Baseline (BM25)	Proposed System	Improvement
BLEU Score (avg.)	0.31	0.43	38.7%
Mean Reciprocal Rank (MRR)	0.62	0.87	40.3%
Avg. Query Latency (sec)	0.8	3.2	—
Answer Faithfulness (%)	N/A	91%	—
Answer Relevance (%)	67%	89%	32.8%

Table 6.3: Performance Benchmarks vs. BM25 Baseline

The proposed system achieves an MRR of 0.87, significantly exceeding the target of 0.80 set in the objectives. The BLEU score improvement of 38.7% over the baseline confirms that the generated answers are more lexically aligned with the reference answers. Answer latency of 3.2 seconds is well within the 5-second requirement. Answer faithfulness—the proportion of generated answers that contain no hallucinated facts not present in the retrieved context—is 91%, demonstrating the effectiveness of the RAG grounding strategy.

CHAPTER 7: SUMMARY AND CONCLUSION

7.1 Summary

This dissertation has presented the complete lifecycle of an LLM-Powered Semantic Search and Question Answering System for Document Intelligence, from initial problem definition through requirements analysis, system design, implementation, and evaluation. The project demonstrates the practical application of Retrieval-Augmented Generation to enable accurate, context-grounded question answering over private document corpora.

The system successfully integrates a sentence transformer embedding model, a persistent vector database (ChromaDB), and the OpenAI GPT API into a cohesive pipeline exposed through an intuitive Streamlit web interface. All primary objectives have been met: the system processes PDF, DOCX, and TXT documents; retrieves semantically relevant passages with an MRR of 0.87; and generates faithful, attributed answers with an average latency of 3.2 seconds.

7.2 Key Findings

21. Semantic retrieval using dense embeddings significantly outperforms keyword-based retrieval (BM25) on the curated benchmark, confirming the hypothesis that embedding-based similarity better captures the semantic intent of natural language queries [3][5].
22. The RAG architecture effectively mitigates the hallucination problem observed in vanilla LLM inference by grounding the generation process in retrieved evidence [2][11].
23. Overlapping chunk strategy with 64-token overlap prevents information loss at chunk boundaries and improves the coherence of retrieved context.
24. System performance is adequate for interactive use cases, with 95th-percentile latency remaining below 5 seconds even under concurrent load.

7.3 Conclusion

The LLM-Powered Semantic Search and QA System represents a significant step towards democratising access to institutional knowledge stored in document form. By combining the semantic richness of transformer-based embeddings with the generative power of large language models, the system delivers a qualitatively superior information access experience compared to conventional approaches. The modular architecture ensures that components can be independently upgraded as the underlying technologies continue to evolve.

The project makes a meaningful contribution to the intersection of Information Retrieval and Natural Language Generation, and its methodology is transferable to diverse domains including legal document analysis, medical record querying, educational content retrieval, and enterprise knowledge management. It aligns directly with SDGs 4, 8, and 9, contributing to quality education, economic productivity, and technological innovation.

CHAPTER 8: LIMITATIONS OF THE PROJECT AND FUTURE WORK

8.1 Limitations

Despite its demonstrated effectiveness, the system has several limitations that should be acknowledged:

Language Support

The current implementation supports English-language documents only. The all-MiniLM-L6-v2 embedding model is optimised for English text and may produce suboptimal embeddings for documents in other languages. Extension to multilingual support would require a multilingual embedding model such as paraphrase-multilingual-MiniLM.

Scanned and Image-Based PDFs

The document loader relies on text extraction and does not perform Optical Character Recognition (OCR). Documents consisting entirely of scanned images cannot be processed. Integration of an OCR engine such as Tesseract would address this limitation.

LLM Cost and Privacy

Reliance on the OpenAI API introduces ongoing costs and raises data privacy concerns for sensitive document corpora, as document content is transmitted to an external service. This can be mitigated by deploying a locally hosted open-source model such as Mistral 7B or LLaMA 3, at the cost of increased infrastructure requirements.

Evaluation Scope

The evaluation was conducted on a relatively small benchmark of 120 question-answer pairs across three documents. A more comprehensive evaluation across diverse domains and document types is necessary to fully characterise system performance.

Context Window Limitations

For very long documents with dense, interconnected information, the fixed number of retrieved chunks (top-k = 5) may not always capture sufficient context for complex, multi-hop questions

[9]. Advanced techniques such as HyDE (Hypothetical Document Embeddings) or re-ranking with a cross-encoder could improve retrieval for such cases [11].

8.2 Future Work

The following directions are proposed for future development:

Multilingual Support

Integrating a multilingual embedding model and extending the preprocessing pipeline to handle non-English documents would significantly broaden the system's applicability, particularly in multilingual institutional environments.

OCR Integration

Adding Tesseract-based OCR to the ingestion pipeline would enable the system to process scanned PDFs and image-based documents, substantially expanding the supported document types.

Advanced Retrieval Techniques

Implementing a two-stage retrieval pipeline—sparse retrieval (BM25) followed by dense reranking using a cross-encoder—would improve precision for complex queries [1][5]. HyDE, which generates a hypothetical answer and uses it to retrieve relevant passages, is another promising direction [11].

Fine-tuned Domain Models

For domain-specific deployments (e.g., legal, medical), fine-tuning the embedding model on domain-specific data using contrastive learning (e.g., via the BEIR benchmark) could yield significant improvements in retrieval accuracy [15].

Conversational Interface

Extending the system to support multi-turn conversations with memory would enable users to ask follow-up questions and receive contextually coherent responses, moving beyond the current single-turn query paradigm.

Federated and On-Device Deployment

Exploring federated architectures where document embeddings are computed and stored on-device, with only query vectors transmitted, would address privacy concerns without sacrificing retrieval quality.

BIBLIOGRAGHY

[1] L. Gao, X. Ma, J. Lin, and J. Callan, "Tevatron: An Efficient and Flexible Toolkit for Dense Retrieval," arXiv preprint arXiv:2203.05765, 2022.

[2] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474, 2020.

[3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proceedings of EMNLP 2019, pp. 3982–3992, 2019.

[4] T. Brown et al., "Language Models are Few-Shot Learners," in NeurIPS 2020, vol. 33, pp. 1877–1901, 2020.

[5] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," in Proceedings of EMNLP 2020, pp. 6769–6781, 2020.

[6] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535–547, 2021.

[7] A. Vaswani et al., "Attention is All You Need," in NeurIPS 2017, vol. 30, 2017.

[8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in NAACL-HLT 2019, pp. 4171–4186, 2019.

[9] S. Wang and J. Zhao, "Multi-hop Question Answering via Reasoning Chains," arXiv preprint arXiv:1910.02610, 2019.

[10] R. Zhu, J. Zhao, H. Liu et al., "ERNIE-ViLG: Unified Generative Pre-training for Bidirectional Vision-Language Generation," arXiv preprint arXiv:2112.15283, 2021.

[11] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," arXiv preprint arXiv:2310.11511, 2023.

[12] LangChain Documentation, "LangChain: Building Applications with LLMs through Composability," [Online]. Available: https://docs.langchain.com. [Accessed: March 2026].

[13] ChromaDB Documentation, "ChromaDB: The Open-Source Embedding Database," [Online]. Available: https://docs.trychroma.com. [Accessed: March 2026].

[14] OpenAI, "GPT-4 Technical Report," arXiv preprint arXiv:2303.08774, 2023.

[15] A. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models," in NeurIPS 2021, 2021.

APPENDIX

Appendix A: Project Directory Structure

```
semantic-qa-system/
├── app.py                    # Streamlit web application
├── requirements.txt          # Python dependencies
├── .env.example              # Environment variable template
├── chroma_store/             # Persistent vector database
├── uploads/                  # Uploaded document storage
├── src/
│   ├── doc_loader.py         # Document loading and format detection
│   ├── preprocessor.py       # Text cleaning and normalisation
│   ├── chunker.py            # Text chunking with overlap
│   ├── embedder.py           # Sentence embedding engine
│   ├── vector_store.py       # ChromaDB wrapper
│   ├── query_engine.py       # Semantic search engine
│   └── answer_generator.py   # LLM answer generation
├── tests/
│   ├── test_loader.py
│   ├── test_embedder.py
│   ├── test_query_engine.py
│   └── test_integration.py
└── README.md
```

Appendix B: Environment Variables

The system requires the following environment variables to be configured in a .env file prior to deployment:

```
OPENAI_API_KEY=sk-...           # OpenAI API key for LLM inference
CHROMA_PERSIST_DIR=./chroma_store  # Directory for vector store persistence
EMBEDDING_MODEL=all-MiniLM-L6-v2   # HuggingFace model identifier
TOP_K_RESULTS=5                     # Number of chunks to retrieve per query
MAX_FILE_SIZE_MB=50                 # Maximum upload file size in megabytes
CHUNK_SIZE=512                      # Token size per chunk
CHUNK_OVERLAP=64                    # Overlap tokens between chunks
```

Appendix C: Sample Query-Answer Pairs from Evaluation Benchmark

Query	Reference Answer
What is the maximum file size supported by the system?	The system supports file uploads up to 50 MB in size.
Which embedding model is used for document encoding?	The all-MiniLM-L6-v2 sentence transformer model is used.
What databases does the system use to store embeddings?	ChromaDB is used as the persistent vector database.
What metrics were used to evaluate the system?	BLEU score, Mean Reciprocal Rank (MRR), and answer latency.
What is the chunk overlap used during text splitting?	A chunk overlap of 64 tokens is used.